OpenClaw Vector Memory

OpenClaw Vector Memory

1. Course Content
    - Course Overview
        - What is Vector Memory
        - How It Works
        - Use Cases
        - Workflow
        - Features
2. Preparation
3. Memory Feature Demo Case
4. Retrieving Memory via Vector Search
    - 4.1 Prerequisites
    - 4.2 Configure embeddings Model API_KEY
    - 4.3 Verify Vector Memory

1. Course Content

Course Overview

OpenClaw has powerful memory capabilities that enable the robot to remember important information during conversations and intelligently retrieve and use these memories in subsequent interactions. This section will introduce OpenClaw's Vector Memory (Semantic Memory) system, which is an intelligent memory retrieval mechanism based on semantic understanding.

What is Vector Memory

Vector Memory is OpenClaw's built-in **semantic search memory system**, which implements intelligent memory retrieval through the following core technologies:

- **Vector Embedding**: Converts memory documents into high-dimensional vector representations, enabling computers to understand the semantic meaning of text
- **Semantic Retrieval**: Matches based on the semantic similarity of queries, not just keyword matching, can understand synonyms and related concepts

How It Works

1. **Vectorization Processing**
    - Converts content from MEMORY.md and memory/*.md into vector representations
    - Supports semantic understanding, not just keyword matching
2. **Semantic Retrieval**
    - Based on the semantic meaning of queries, not exact keywords
    - Returns the most relevant text fragments and their location information

Use Cases

Scenario	Example
Recalling Past Decisions	"The decision about sorting order before"
Finding Related Experience	"How to handle robotic arm grasping failure"
Task Context	"The project the user mentioned last week"
Personal Preferences	"User's preferred interaction method"

Workflow

```
User Question → memory_search(Semantic Query) → Returns Relevant Fragments → memory_get(Read Specific Location) → Generate Response
```

Features

- **Intelligent Matching**: Understands synonyms and related concepts
- **Automatic Citation**: Results include file path and line number
- **Complementary to File Memory**: Vector memory for retrieval, file memory for persistent storage

2. Preparation

- First study the 【OpenClaw Skills Development】 chapter to master creating new skills

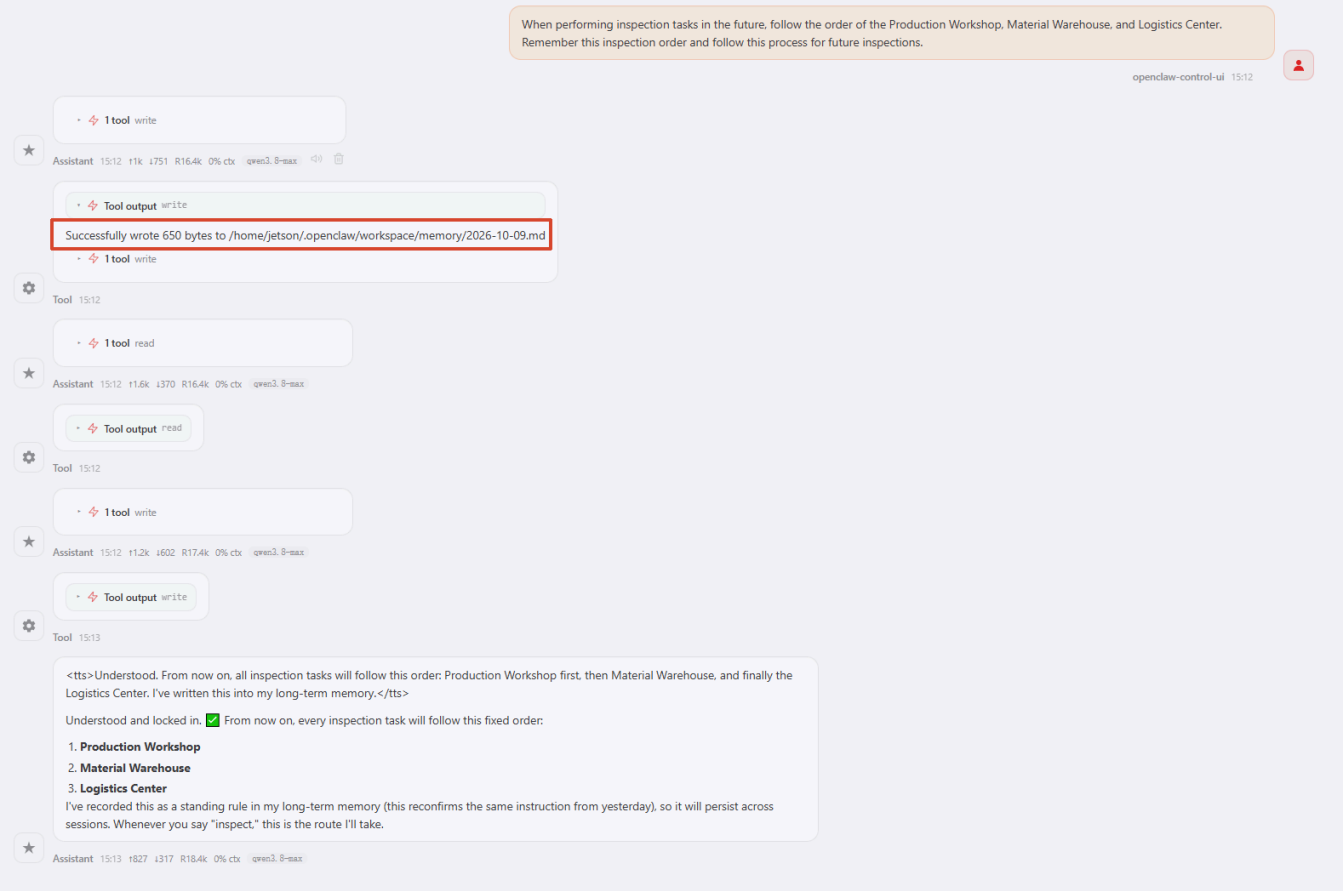
```
ros2 launch yahboomcar_bringup mcp_service.launch.py use_camera:=true
```

- If you need voice replies, you need to add the use_tts parameter: `ros2 launch yahboomcar_bringup mcp_service.launch.py use_camera:=true use_tts:=true`, otherwise it can be omitted

3. Memory Feature Demo Case

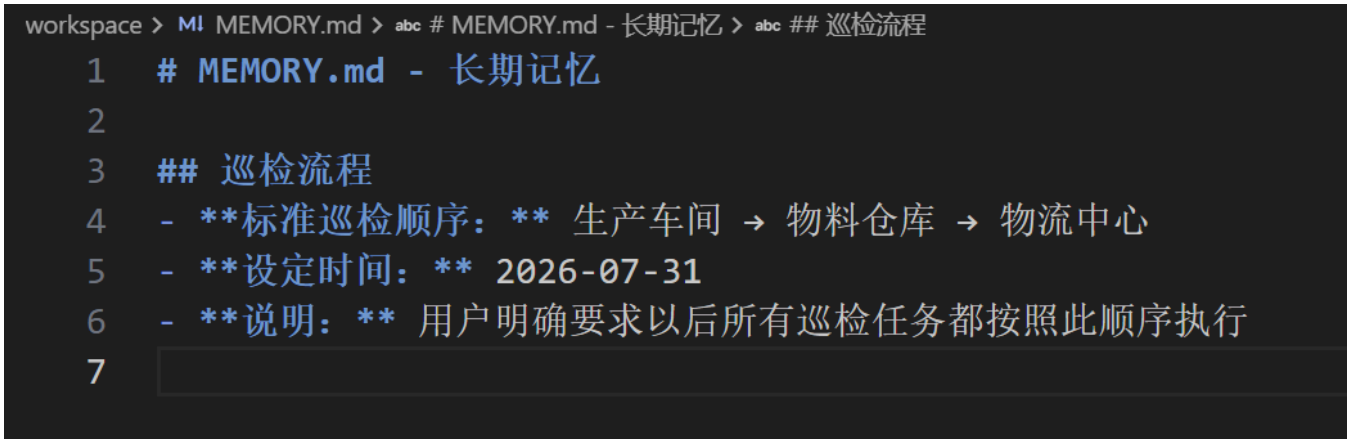
> **Tip**
>
> - This case is for demonstration reference only, you can have free conversations based on your own needs. The following is a case demonstration using WebChat as an example

- When performing inspection tasks in the future, follow the order of production workshop, material warehouse, and logistics center. Remember this inspection order and follow this process for future inspections.



- OpenClaw's memory content will be stored in files at the following path. You can view the memory content from the above case:

```
cat $HOME/.openclaw/workspace/MEMORY.md
```



4. Retrieving Memory via Vector Search

- OpenClaw supports slicing and retrieving memory documents in vector mode, suitable for semantic retrieval of memories in large-scale memory documents

> **Tip**
>
> - OpenClaw's built-in default mode is **full-text search mode**, suitable for the vast majority of use

> cases. If you have higher requirements for the memory feature, you can refer to the following tutorial to implement vector storage of memory documents by adding an external **embeddings** model

4.1 Prerequisites

- You need to register an account with an **embeddings** model provider. The following demonstration uses the **openrouter** platform as an example. For account registration and API application, refer to the tutorial 【AI Model Development—Register Model Service Account—3. openrouter Platform Account】

4.2 Configure embeddings Model API_KEY

- Set the model provider (openai interface protocol)

```
openclaw config set agents.defaults.memorySearch.provider "openai"
```

- Set the baseUrl. Here using openrouter as an example. If you are using **embeddings** models from other providers, replace with the actual baseUrl

```
openclaw config set agents.defaults.memorySearch.remote.baseUrl
"https://openrouter.ai/api/v1"
```

- Set the embeddings model. Here using the `"openai/text-embedding-3-small"` model as an example. If you are using other **embeddings** models, replace with the actual name

```
openclaw config set agents.defaults.memorySearch.model "openai/text-embedding-3-small"
```

- Set the API_KEY for the **embeddings** model

```
openclaw config set agents.defaults.memorySearch.remote.apiKey "sk-or-v1-
133949f96bec556343539a587f53d077xxxxxxxxxxxxxxxx"
```

- Restart the OpenClaw gateway to take effect

```
openclaw gateway restart
```

Configuration Parameter Description:

- `agents.defaults.memorySearch.provider`: The model provider interface protocol for memory search. Setting it to `"openai"` means using an OpenAI-compatible interface
- `agents.defaults.memorySearch.remote.baseUrl`: The API service address of the embeddings model. Fill in the corresponding baseUrl according to the provider actually used
- `agents.defaults.memorySearch.model`: The name of the embeddings model, used to convert memory documents into vector representations. Different providers offer different embeddings models, so fill in according to the model actually used
- `agents.defaults.memorySearch.remote.apiKey`: The API key for the embeddings model, used for

identity authentication

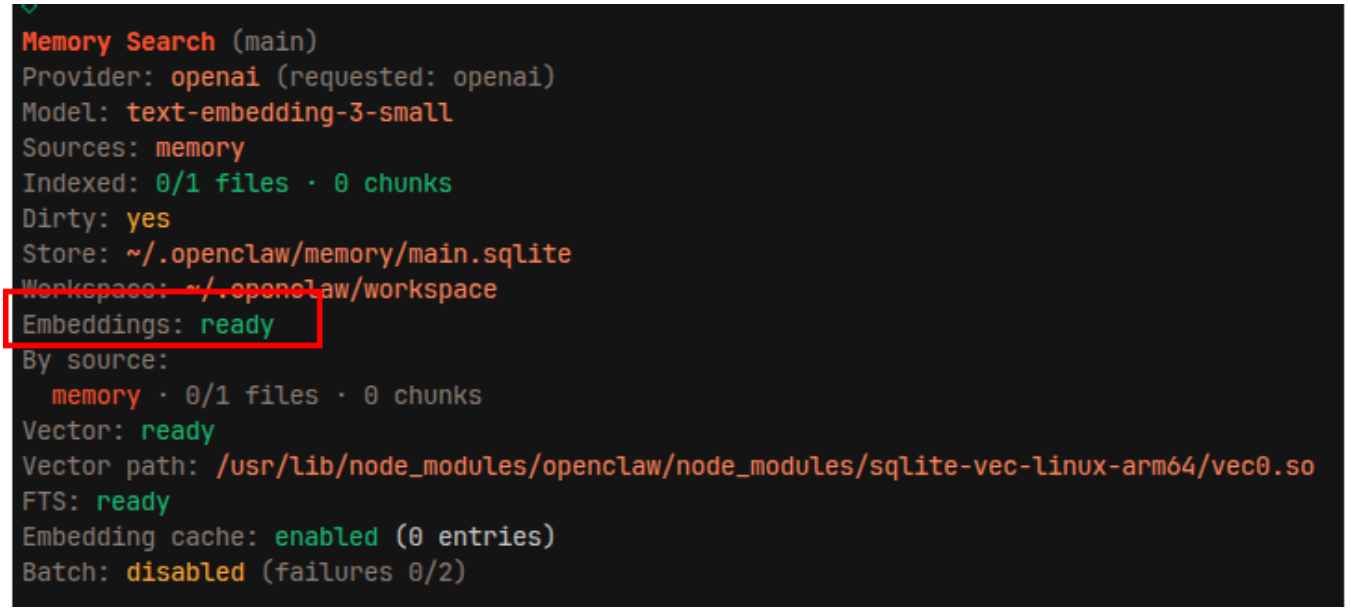
CLI Command Description:

- `openclaw memory status --deep`: Views the complete status of vector memory, including the sliced file paths and vector index information, used to confirm whether the configuration is in effect
- `openclaw memory index --force`: Forces a rebuild of the memory index. It needs to be executed manually once after initial configuration or memory document updates
- `openclaw memory search "keyword"`: Tests semantic retrieval, returning the memory fragments most semantically relevant to the keyword along with scores. A higher score indicates higher semantic similarity (typically > 0.3 indicates good relevance)

4.3 Verify Vector Memory

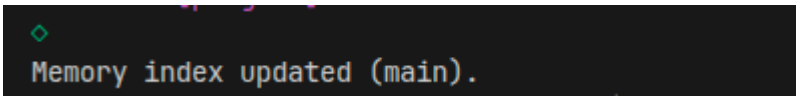
- Check if vector memory is configured and active. The following output proves the configuration is successful, where Vector is the path of the sliced file

```
openclaw memory status --deep
```



- After initial configuration, you can manually update the index once

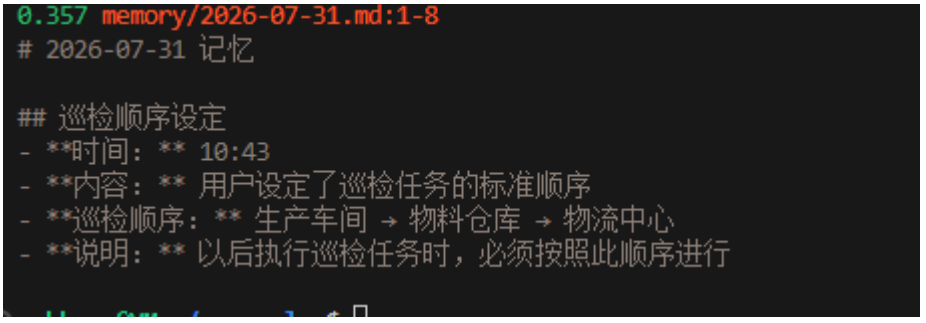
```
openclaw memory index --force
```



- Test whether memory retrieval can find the memory fragments from the demonstration case above

```
openclaw memory search "inspection"
```

- Here `0.357` is the semantic relevance score. A higher score indicates higher semantic similarity. Results greater than 0.3 typically have good relevance


0.357 memory/2026-07-31.md:1-8
2026-07-31 记忆
巡检顺序设定
- **时间：** 10:43
- **内容：** 用户设定了巡检任务的标准顺序
- **巡检顺序：** 生产车间 → 物料仓库 → 物流中心
- **说明：** 以后执行巡检任务时，必须按照此顺序进行